

BUSINESS ANALYTICS
INDIVIDUAL CASE STUDY REPORT

**DefectSignal: Prioritizing Urgent Product Complaints
in Consumer Tech Reviews**

Srihitha Thotakura
Roll No: CB.SC.U4CSE23450 | CSE-E
B.Tech Computer Science and Engineering
Amrita Vishwa Vidyapeetham, Coimbatore

1. Problem Statement and Objectives

1.1 Problem Statement

Tech brands receive thousands of product reviews across platforms like Google Play Store, Flipkart and Amazon, but quality and support teams typically review this feedback manually and in batches, spotting defect patterns only after complaints have already escalated into returns, poor ratings, or reputational damage. This delay is costly, as it allows genuine safety and quality issues to go unnoticed until they have already caused customer churn, warranty losses, and brand damage. This study focuses on portable consumer electronics (wireless earbuds, smartwatches, and power banks), where faults such as overheating and battery failure can also be safety risks.

Solving this problem is important because early identification of urgent complaints allows quality assurance (QA) and customer support teams to intervene proactively rather than reactively. Product managers, QA teams, and customer support functions stand to benefit directly, as would the wider business through improved customer retention and reduced warranty costs. Data analytics can contribute by applying text classification techniques to review data, enabling automatic detection and prioritization of urgent, defect-related complaints amid the large volume of routine feedback that teams receive daily.

1.2 Objectives

- Classify consumer reviews as Urgent (indicating safety risk, malfunction, or refund/replacement demand) or Non-Urgent (general feedback) using TF-IDF with Logistic Regression and Naive Bayes, evaluated mainly on recall and F1-score for the Urgent class.
- Identify recurring language patterns and product features most associated with urgent complaints across the three product types.
- Build a data-driven triage framework that helps QA and customer support teams prioritize high-risk reviews for faster response.

2. Data Collection and Dataset Description

2.1 Collection Method and Sources

The dataset was built by web scraping publicly accessible customer reviews of portable consumer electronics (wireless earbuds, smartwatches and power banks) from two platforms in the week of 18–23 September 2026, based on the most recent review timestamps in the collected data:

- Google Play Store: reviews of five companion apps used with smartwatches and earbuds — boAt Crest, NoiseFit, Zepp (Amazfit), realme Link and HeyMelody (OnePlus/OPPO). These reviews often describe device faults such as connection, syncing and charging problems.
- Flipkart: reviews of 12 popular product listings (5 power banks, 4 earbuds, 3 smartwatches) from brands including boAt, Noise, Mi, Ambrane, Portronics, Mivi, OnePlus and Fire-Boltt.

No ready-made dataset was used. Amazon.in, named in the proposal as an optional source, was not used: it shows only a small number of reviews per product to visitors who are not logged in, so it would have added negligible volume without bypassing login.

2.2 Collection Procedure

Google Play. A Python script using the open-source google-play-scraper library collected reviews from the India store (English). For each app it requested up to 3,000 reviews, split between the “newest” and “most relevant” sort orders, with a 1–2.5 second pause between requests and automatic retries on errors. Duplicates within an app were removed.

Flipkart. Reviews were collected with the Apify actor Flipkart Reviews Scraper (solidcode/flipkart-scraper), using the product page URL of each listing. Four runs were made:

Run	Products	Sort order	Reviews / product	Rows
Power banks	5	Most recent	up to 200	1,000
Earbuds	4	Most recent	up to 140	560
Smartwatches	3	Most recent	up to 180	538
All products	12	Most helpful	up to 30	360

The 2,458 rows collected contained 26 overlapping reviews, so 2,432 unique Flipkart reviews remained. Products were selected as widely reviewed listings from popular brands in India.

Consolidation. A Python script merged all sources into one file with a common structure. It converted Flipkart’s relative dates (“3 months ago”) to approximate dates and dropped all personal and identifying fields.

2.3 Dataset Overview

The final raw dataset (reviews_raw.csv) contains 16,768 reviews.

Product category	Google Play	Flipkart	Total
Smartwatch	8,585	623	9,208
Earbuds	2,758	680	3,438
Wearable (mixed, realme Link)	2,993	–	2,993
Power bank	–	1,129	1,129
Total	14,336	2,432	16,768

- Time span: 26 September 2018 to 22 September 2026, with most reviews from 2026. Flipkart shows relative dates, so 85% of Flipkart dates are approximate.
- Ratings: 41.0% are 5-star and 29.8% are 1-star, so ratings are strongly polarized. The average rating is 3.18 on Google Play and 3.85 on Flipkart.

2.4 Attributes

The file has 13 attributes.

Attribute	Type	Description
review_text	Text	Full review written by the customer
review_title	Text	Review headline (Flipkart only)
rating	Integer (1–5)	Star rating given by the reviewer
review_date	Date	Date of the review (approximate for Flipkart)
date_approx	Boolean	Whether the date is approximate
platform_source	Category	Google Play Store or Flipkart
source_type	Category	app (companion-app review) or product (product review)
product_category	Category	Smartwatch, earbuds, power bank or mixed wearable
brand	Category	Brand name
product_name	Text	Product or app name
helpful_votes	Integer	Thumbs-up (Play) or helpful count (Flipkart)
certified_buyer	Boolean	Flipkart’s verified-purchase flag (Flipkart only)
app_id	Text	Play Store app identifier (Play only)

The Urgent/Non-Urgent label and text-length features are created later, in Section 3.

2.5 Important Variables

- review_text: the input to the text classification model.
- Urgency label (Urgent or Non-Urgent): the target variable, derived from the review text as described in Section 3.
- rating, product_category, platform_source: used to compare urgent-complaint patterns across segments and to check label quality.

2.6 Privacy, Ethics and Limitations

Only public review text, ratings, dates and product details were collected. No login was used. Reviewer names, locations, profile links, review IDs, review URLs and images were not stored, and any phone numbers or emails typed inside reviews are masked during cleaning. Requests were paced to avoid overloading the websites, and the data is used only for this academic study.

Limitations to keep in mind:

- The sample is not random. It follows each platform’s “newest”, “most relevant” and “most helpful” orderings.
- Google Play makes up 85.5% of the data, and its reviews concern apps as well as devices. This is why source_type is kept as a variable.
- Power banks are represented only by Flipkart reviews (1,129, or 6.7%).
- Flipkart covers just 12 products, and the dataset is English-language reviews only.

3. Data Preparation and Exploratory Analysis

3.1–3.2 Loading and Data Quality Checks

The raw file combines Google Play companion-app reviews (source_type = app) and Flipkart product reviews (source_type = product). Before cleaning, a quality scan flagged the issues a text classifier is most sensitive to: 27.3% of reviews are only 1–2 words long; 21.7% of texts are exact duplicates, dominated by generic praise such as “good” (989 occurrences), “nice” (376) and “very good” (153); 23 reviews contain un-decoded HTML entities; and 180 contain embedded line breaks. Duplicates matter because the same text could otherwise land in both the training and test sets and inflate reported accuracy. Flipkart reviews are also far shorter than Google Play reviews (median 6 words vs. 33 words), reflecting the more casual, rating-driven review culture on that platform.

3.3 Data Cleaning

Each cleaning rule is logged so the effect on the dataset is traceable:

Step	Rule	Reason
1	Decode HTML entities, collapse whitespace, mask phone numbers, e-mails and URLs	Consistent text; removes personal details typed inside reviews
2	Drop reviews with no letters (emoji/symbols only)	No text to classify
3	Drop reviews written mostly in non-Latin scripts	Scope is English-language reviews
4	Drop reviews mostly in an Indian language written in English letters	Word features are English-only
5	Drop exact duplicate texts (case/punctuation/emoji-insensitive)	Prevents train/test leakage and repeated noise

Short reviews (1–2 words) were kept deliberately: a real triage tool receives them, and they are valid Non-Urgent training examples. After cleaning, 12,449 of the 16,768 raw reviews remained (74.2% retention).

3.4 Exploratory Analysis Before Labeling

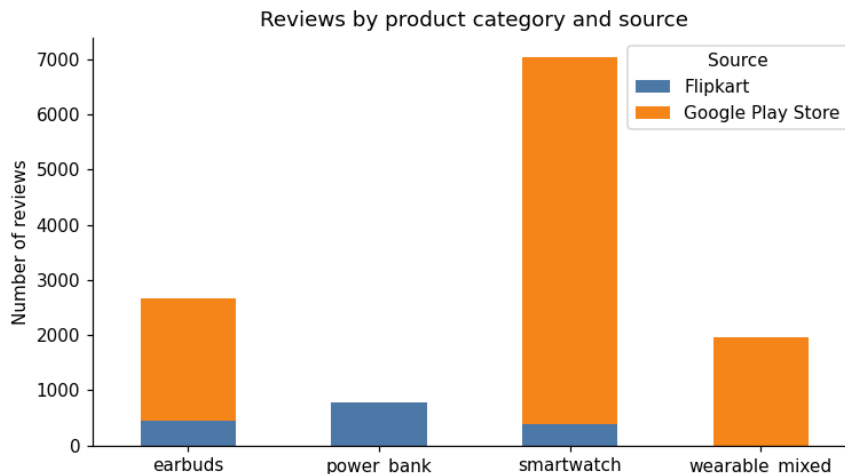


Figure 1. Reviews by product category and source (post-cleaning).

Smartwatches dominate the cleaned dataset (7,044 reviews, mostly Google Play companion-app reviews), followed by earbuds (2,663), the mixed-wearable realme Link app (1,967) and power banks (775, Flipkart only). Star ratings are strongly polarized: 41.0% of raw reviews are 5-star and 29.8% are 1-star, with comparatively few 2–4-star reviews — typical of consumer review platforms, where reviewers tend to write only when very satisfied or very frustrated.

3.5 Creating the Urgent / Non-Urgent Label

Rather than hand-labeling all 12,449 reviews, transparent keyword/pattern rules were used to weakly label the training data, organized into eight rule types covering the ways an urgent complaint is typically expressed: app/device connectivity failures, general “not working” statements, functional malfunctions, refund or fraud language, unresolved support complaints, hardware defects, battery failure, and safety hazards (e.g. overheating). Overall, 21.9% of cleaned reviews were rule-labeled Urgent.

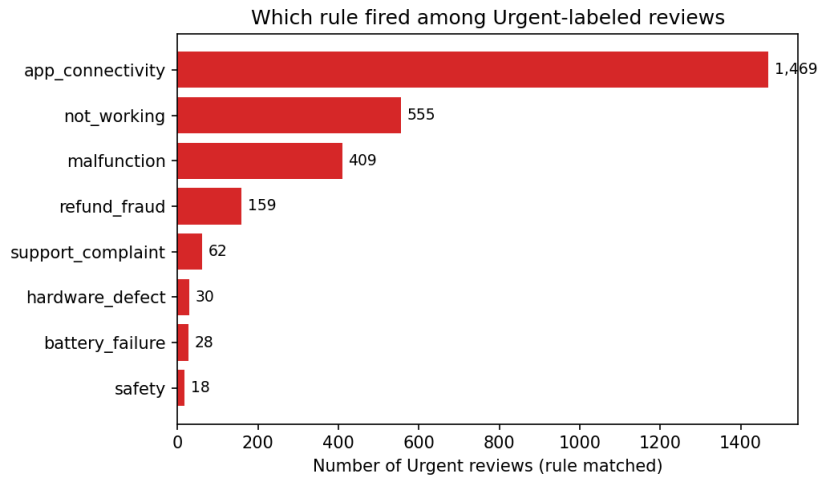


Figure 2. Which rule fired among rule-labeled Urgent reviews.

App/device connectivity problems dominate the Urgent label (1,469 of 2,730 rule matches, 53.8%), followed by general “not working” complaints (555) and functional malfunctions (409). Only 18 matches were safety-specific (e.g. overheating language), reflecting that most urgency in this dataset is software/pairing reliability rather than physical hazard — an important nuance for how the results should be read in Section 6.

3.6 Gold Sample: Checking Label Quality

A random sample of 700 cleaned reviews (5.6% of the dataset) was hand-labeled Urgent/Non-Urgent by the author, split into a 280-review development set and a 420-review held-out test set. The gold sample is 18.7% Urgent by human judgment, close to the 21.9% rate from the automated rules.

Comparing the rule-based labels against the human labels on the TEST portion (the part never used to adjust the rules) gives: precision 0.58, recall 0.65, F1 0.61, and Cohen’s kappa 0.50 — “moderate” agreement beyond chance. In practice this means the rules over-flag some reviews that a human would call routine (26 false alarms in the reviewed set, often ambiguous connectivity complaints or one-line “not working” reviews without further detail) and under-flag some genuinely urgent ones that use indirect language (13 misses, e.g. reviews about a broken strap, buggy updates, or being forced to create an account, which read as frustrated but do not match a keyword pattern). This kappa of 0.50 is an important ceiling: since the classifier in Section 4 is trained on these noisy rule-based labels, its performance cannot exceed what a moderately-agreeing labeling process allows.

3.7 Text Preprocessing for the Classifier

For the TF-IDF features used in Section 4, review text was lowercased, stripped of the masked-entity placeholders, and left otherwise close to its original form (unlike the keyword-rule step, common stop words are removed here since they carry little weight once TF-IDF-weighted). No reviews became empty after preprocessing.

3.8 Exploratory Analysis of the Labeled Data

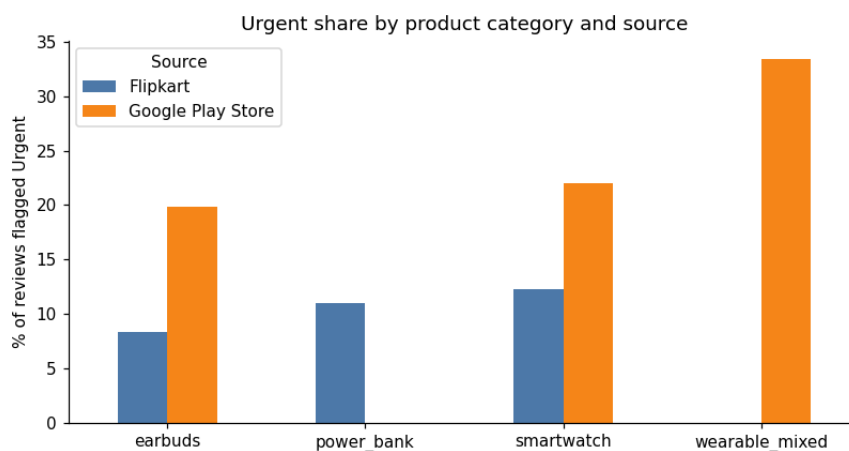


Figure 3. Urgent share by product category and source.

Urgent share varies sharply by segment. By star rating, 42.9% of 1-star reviews are Urgent versus just 0.5% of 5-star reviews — but a notable 20.3% of 3-star reviews are also Urgent, showing that star rating alone is an unreliable triage signal for the middle of the distribution. By platform, Google Play reviews are far more often Urgent than Flipkart reviews (23.6% vs. 10.6%), plausibly

because Google Play reviews concern the companion app itself (where connectivity bugs live) while Flipkart reviews are shorter, more general product reviews.

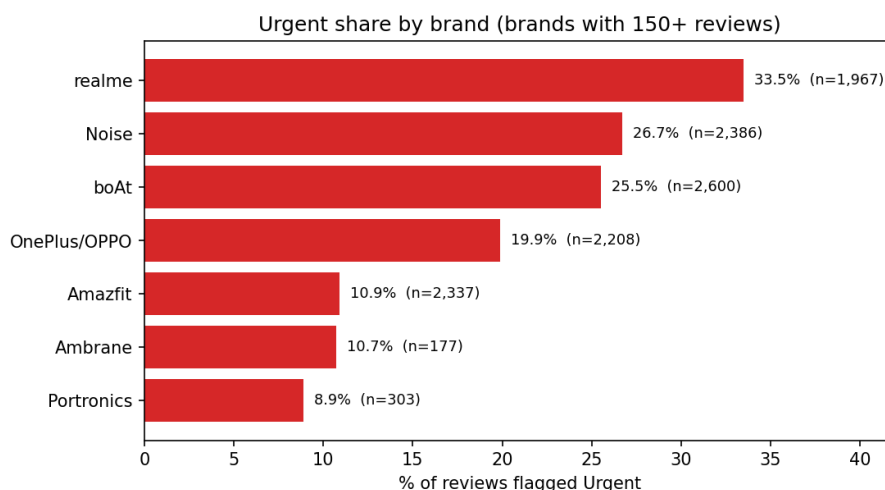


Figure 4. Urgent share by brand (brands with 150+ reviews).

Among brands with substantial review volume, realme (33.5%) and Noise (26.7%) show markedly higher Urgent shares than Amazfit (10.9%), Ambrane (10.7%) and Portronics (8.9%), pointing to where QA attention could be prioritized (Section 6 revisits this).

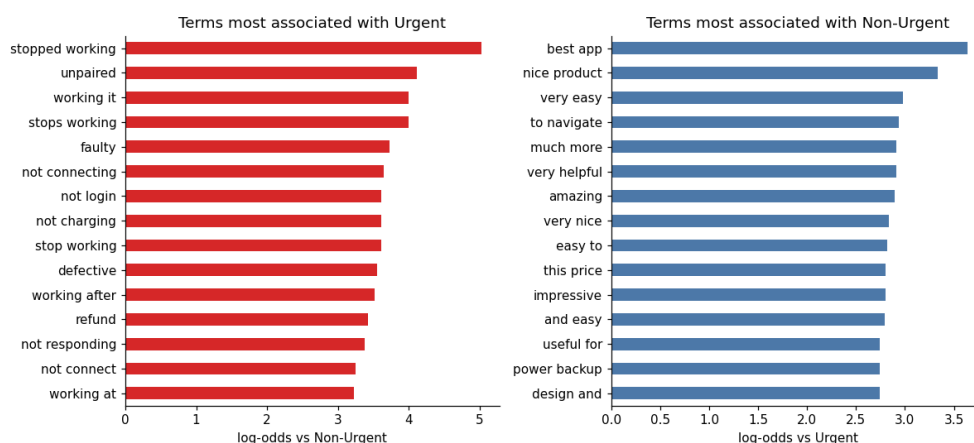


Figure 5. Terms most associated with each label under the weak-label rules (log-odds).

3.9 Train/Test Split and Export

The final labeled file (reviews_labeled.csv, 12,449 rows) was split into three parts, each used for a distinct purpose in Section 4:

- Train: 11,749 reviews, rule-based labels (21.9% Urgent) — used to fit the models.
- Dev: 280 reviews, human labels (16.8% Urgent) — used to choose between models.
- Test: 420 reviews, human labels (20.0% Urgent) — touched exactly once, for the final honest evaluation.

4. Analytics Method and Implementation

4.1 Method Selection and Justification

TF-IDF (Term Frequency–Inverse Document Frequency) with linear classifiers was chosen as the analytics method, consistent with the text-classification techniques covered in the Business Analytics syllabus. TF-IDF is well suited to this problem because review text is short, high-dimensional and sparse, and because the resulting linear models (Logistic Regression, Naive Bayes) remain interpretable — a QA team can see exactly which words push a review toward “Urgent” (Section 4.5), which matters for trust in an automated triage tool. Both algorithms were compared rather than assuming one in advance, since Logistic Regression typically gives better-calibrated probabilities while Naive Bayes often has an edge with sparse, small feature counts.

4.2 Feature Engineering

Features were unigrams and bigrams, English stop words removed, terms appearing in at least 5 training reviews, capped at 20,000 terms. The vectorizer was fit on the training split only, and the dev/test sets were transformed with that same vocabulary so no evaluation-set information leaks into the features. In practice, the min_df=5 threshold on 11,749 training reviews produced a vocabulary of 7,529 terms, comfortably under the 20,000 cap.

4.3 Model Training and Selection (Dev Split)

Both models used class weighting (class_weight="balanced" for Logistic Regression; per-sample weighting for Multinomial Naive Bayes, which has no native class_weight parameter) to counter the roughly 4-to-1 Non-Urgent-to-Urgent imbalance in the rule-labeled training data. Because missing a genuine Urgent complaint is the costlier business error (Section 4.4 discusses why), model selection favored Urgent-class F1 and recall over overall accuracy.

Model (dev)	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.807	0.446	0.617	0.518	0.863	0.517
Naive Bayes	0.771	0.411	0.830	0.549	0.845	0.531

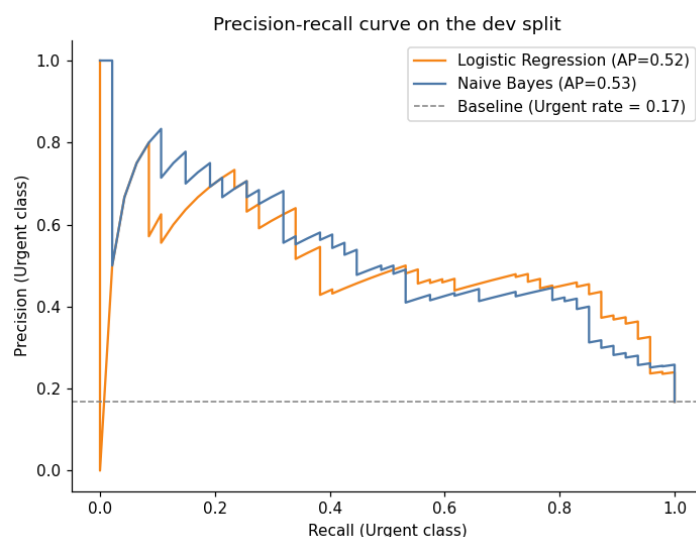


Figure 6. Precision-recall curves on the dev split (Urgent class).

Naive Bayes was selected: despite lower overall accuracy, it reaches substantially higher Urgent-class recall (0.830 vs. 0.617) and a better F1 (0.549 vs. 0.518) on the dev split, which matches the project’s stated priority of not missing genuine defect/safety complaints.

4.4 Final Evaluation on the Test Split

The test split was evaluated exactly once, after model selection was locked in on dev. Both models are reported for transparency, but the selected model (Naive Bayes) is the project’s headline result:

Model (test)	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.810	0.517	0.738	0.608	0.876	0.616
Naive Bayes (selected)	0.781	0.473	0.845	0.607	0.880	0.656

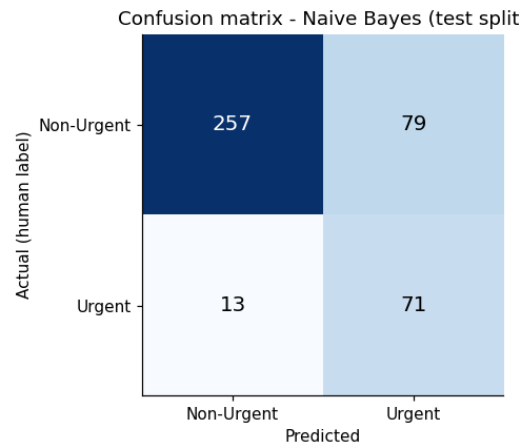


Figure 7. Confusion matrix — Naive Bayes, test split (n=420).

Of 84 genuinely Urgent test reviews (by human label), the selected model correctly flags 71 (recall 0.845) and misses 13. Of the 336 genuinely Non-Urgent reviews, it correctly clears 257 and raises 79 false alarms. Given the business framing in Section 1 — a missed Urgent review is the costly error, a false alarm only costs a QA reviewer a few minutes — this recall-precision trade-off is the intended one.

4.5 What the Model Learned: Top Terms per Class

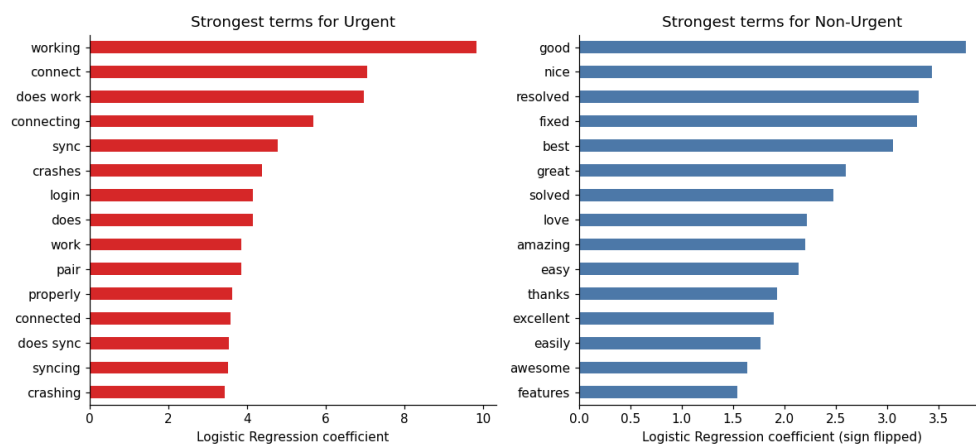


Figure 8. Strongest Logistic Regression coefficients for each class.

The strongest Urgent-associated terms — “working”, “connect”, “does work”, “connecting”, “sync”, “crashes”, “login” — confirm the pattern already visible in the rule-type breakdown (Figure 2): the model has learned that connectivity and pairing failures are the dominant urgency signal, addressing Objective 2. The strongest Non-Urgent terms (“good”, “nice”, “resolved”, “fixed”, “best”) are generic positive/closure language, as expected.

4.6 Error Analysis on the Test Split

Of the 13 false negatives, most are Urgent complaints phrased indirectly — e.g. requests for a replaceable strap, frustration with a firmware update that regressed features, or an app forcing account creation — language that does not strongly overlap with the connectivity/malfunction vocabulary the model relies on. Of the 79 false positives, most contain “not working” language attached to a resolved or minor issue (e.g. a login error later fixed, or a single-line “not working” review with no further detail) — exactly the ambiguous cases where the rule-based training labels themselves disagreed with human judgment in Section 3.6 (kappa 0.50). This is the honest interpretation: model performance is bounded by training-label quality, and the reported test numbers are not overly optimistic since the test split was evaluated exactly once against human labels.

5. Comparison with State-of-the-Art Methods

Recent work on identifying defective or hazardous products from online reviews and user reports falls into three broad approaches: interpretable psycholinguistic feature models, weak/distant-supervision models trained on noisy labels, and transformer-based few-shot classifiers. The table below compares four recent, verifiable studies against this case study’s TF-IDF + Naive Bayes / Logistic Regression urgency-triage approach.

Published Study / Year	Dataset	Method Used	Evaluation Metric	Key Result	Relation to This Study
Abbas & Malik (2023), Electronic Commerce Research	Amazon reviews crawled across 4 product categories	Psychological, linguistic and discrete-emotion features (not TF-IDF) selected via correlation/information gain, fed to ML classifiers	Accuracy	Best model: 84% accuracy distinguishing defective from non-defective reviews	Both target defect-related complaints, but this study engineers affect/psycholinguistic markers rather than TF-IDF n-grams. Our task is finer-grained (Urgent vs. Non-Urgent, not just defective/non-defective) and is evaluated on Urgent-class recall/F1 (0.845 / 0.607) rather than accuracy, since missing an urgent complaint is the costlier error for QA. Different categories, region and label definitions mean the 84% figure is not directly comparable to our 0.781 accuracy.
Mangnoesing, Truşcă & Frasincar (2020), NLDB / LNCS 12089	4,470 Google Play reviews of the Evernote app, scraped by the authors; ~46% hand-annotated	Lexico-semantic pattern learning (genetic programming) vs. a distantly-supervised SVM trained on noisy, pattern-generated labels	Precision / Recall / F1	Distantly-supervised SVM reached F1 $\approx$ 0.47 for defect-report detection, close to the pattern-based approach	Methodologically the closest parallel to our own pipeline: both use noisy/weak labels from rules or patterns to train a classifier, then check against hand-labeled data. Our weak-label agreement (test kappa 0.50, F1 0.61 vs. human labels) is similar in spirit to their distant-supervision gap. Our final classifier's test F1 for Urgent (0.607) is notably higher than their reported ~0.47, though the tasks, platforms and label definitions differ enough that this is suggestive rather than a strict comparison.
Fuchs, Jadhav, Jaishankar, Cauffman & Spanakis (2023), ICAIL '23	3,000 electronic-product reviews, 755 linked to EU Safety Gate hazard reports	Correlation-Coefficient scoring heuristic vs. supervised ML classifiers for hazard detection	ROC-AUC	AUC up to ~0.80 identifying reviews referencing a hazardous product	Closest in spirit to our “Urgent” label, since safety risk is one of its components (only 0.7% of our rule matches were safety-specific, however — most of our Urgent signal is connectivity/malfunction, not physical hazard). Their positive class is anchored to real

Published Study / Year	Dataset	Method Used	Evaluation Metric	Key Result	Relation to This Study
					regulatory recall records; ours is anchored to 700 hand-labeled reviews (kappa 0.50), a weaker but more feasible ground truth for a student project. Our selected model's test ROC-AUC (0.880) is comparable to or slightly above their reported ceiling of ~0.80, though again the label definitions and product scope differ.
Silva et al. (2025), arXiv:2503.16614	341 simulated/forum user reports across 8 hardware-component classes	Zero-/one-/few-shot transformer models (BART, DeBERTa, MiniLM, MPNet) — no full fine-tuning	Accuracy, F1	Best few-shot model (6-layer MiniLM): 0.79 accuracy and F1	Both classify user-written fault reports, but this is multi-class component identification on a tiny dataset using few-shot transformers, while ours is a binary urgency triage on a much larger set (16,768 raw / 700 gold-labeled) using classical TF-IDF + linear models — cheaper to train and easier for a QA team to audit, though it cannot name the specific faulty component the way their multiclass model does. Their finding that a smaller model generalized better on noisy, limited data echoes our own limitation: training labels come from noisy rules (kappa 0.50), which caps what any model trained on them can achieve.

Two additional candidates — a TF-IDF + SVM sentiment study on generic e-commerce reviews, and a multi-classifier study on women's clothing reviews — were considered but not included: both are TF-IDF-based text classification, but on generic sentiment rather than defect/urgency detection, making them a weaker methodological match than the four studies above, all of which specifically target defect, fault or hazard identification from user-generated text.

6. Results, Business Insights and Recommendations

6.1 Headline Result

The selected model (TF-IDF + Multinomial Naive Bayes, class-weighted) reaches, on the held-out human-labeled test split: 78.1% accuracy, 84.5% recall and 47.3% precision on the Urgent class, F1 of 0.607, and ROC-AUC of 0.880. In business terms: of every 100 genuinely urgent reviews, roughly 85 are caught by the model, at the cost of routing an extra 79 harmless reviews to the same queue (out of 336 test-set Non-Urgent reviews) — a deliberate trade-off given that a missed defect/safety complaint is the more expensive error for a QA team.

6.2 Business Insights

- Star rating alone is an unreliable triage signal: while 42.9% of 1-star reviews are Urgent, so are 20.3% of 3-star reviews — a segment a ratings-only alert system would likely deprioritize.
- App/device connectivity failures, not physical hardware defects, dominate urgency: 53.8% of rule-flagged Urgent reviews are connectivity complaints, versus only 0.7% (18 of 2,730) that are safety-specific (e.g. overheating). The urgent-review pipeline is currently mostly a software/pairing-reliability signal, and framing it purely as a “safety” tool would overstate what it detects.
- Google Play (companion-app) reviews are more than twice as likely to be Urgent as Flipkart (product) reviews (23.6% vs. 10.6%) — plausibly because app reviews concern the software layer where bugs live, though this is confounded with `source_type` and cannot be read as “Flipkart products are safer.”
- Brand-level Urgent share varies widely among high-volume brands: realme (33.5%) and Noise (26.7%) run 3× higher than Amazfit (10.9%), Ambrane (10.7%) and Portronics (8.9%), a useful signal for prioritizing QA and companion-app engineering attention.

6.3 Recommendations

- Deploy the Naive Bayes model as a first-pass triage filter that routes predicted-Urgent reviews into a QA/support queue, ranked by predicted probability so the highest-confidence cases are reviewed first.
- Keep a human in the loop: at 47.3% precision, roughly half of flagged reviews are false alarms, so the model should surface candidates for review rather than auto-close or auto-escalate tickets.
- Prioritize engineering attention on app connectivity and pairing reliability, the single largest driver of urgency (53.8% of flagged complaints), ahead of hardware-specific fixes.
- Track brand/companion-app Urgent share as a recurring QA scorecard metric, using the gap between high-share brands (realme, Noise) and low-share brands (Amazfit, Portronics) as a starting benchmark.
- Periodically expand the hand-labeled gold sample and retrain, since the training labels are rule-based and only moderately agree with human judgment (kappa 0.50) — this is the main lever for improving the model beyond its current ceiling.

6.4 Limitations

- Training labels are weak/rule-based (kappa 0.50 vs. human judgment on the test portion), which caps achievable model performance regardless of the classifier used.
- The sample follows each platform’s default sort orders rather than random sampling, and Google Play makes up 85.5% of the data.
- Power banks are represented only by 775 Flipkart reviews, limiting confidence in power-bank-specific findings.
- The 420-review test set means the reported Urgent-class metrics (based on 84 positive cases) carry meaningful sampling uncertainty.

7. Conclusion and References

7.1 Conclusion

This case study built an end-to-end pipeline — from web-scraped review collection through weak labeling, hand-validated gold labels, and TF-IDF text classification — to triage urgent product complaints across wireless earbuds, smartwatches and power banks. The selected Naive Bayes model catches 84.5% of genuinely urgent reviews on a held-out, human-labeled test set, at a precision of 47.3%, and reveals that the dominant driver of urgency in this dataset is companion-app connectivity failure rather than physical hardware defects. Compared against four recent published studies on defect and hazard detection from reviews, the approach here trades the sophistication of psycholinguistic features or transformer few-shot learning for a simpler, fully interpretable and cheaply retrainable pipeline — appropriate given the moderate quality of the underlying weak labels (kappa 0.50), which is the main constraint on how much further performance can be pushed without more hand-labeled data.

7.2 References

1. Abbas, Y., & Malik, M. S. I. (2023). Defective products identification framework using online reviews. *Electronic Commerce Research*, 23(2), 899–920. <https://doi.org/10.1007/s10660-021-09495-8>
2. Fuchs, M., Jadhav, A., Jaishankar, A., Cauffman, C., & Spanakis, G. (2023). “What’s wrong with this product?” – Detection of hazardous products from online reviews. In *Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law (ICAAIL ’23)* (pp. 397–401). ACM.
3. Mangnoesing, G. V. H., Truşcă, M. M., & Frasincar, F. (2020). Pattern learning for detecting defect reports and improvement requests in app reviews. In *Natural Language Processing and Information Systems (NLDB 2020)*, Lecture Notes in Computer Science, vol. 12089. Springer. https://doi.org/10.1007/978-3-030-51310-8_12
4. Silva, M. L. M., Mendonça, A. L. C., Neto, E. R. D., Chaves, I. C., Brito, F. T., Farias, V. A. E., & Machado, J. C. (2025). Classification of user reports for detection of faulty computer components using NLP models: A case study. *arXiv:2503.16614*.
5. google-play-scraper (Python package). PyPI. <https://pypi.org/project/google-play-scraper/>
6. Flipkart Reviews Scraper (Apify actor, solidcode/flipkart-scraper). <https://apify.com/solidcode/flipkart-scraper>
7. Google Play Store. <https://play.google.com>
8. Flipkart. <https://www.flipkart.com>